实验 1　初识 Xinu

截止时间：2026 年 4 月 1 日 23:59

提交要求：

- 正常提交→OBE 上“实验 1”；
- 截止时间后修改后提交→OBE 上新开条目（待 OBE 上创建）。

实验报告编制要求：

- 文件名：学号姓名.pdf　　←注意
- 报告标题：体现出“实验 1”等信息
- 报告作者：学号　姓名　　←注意
- 不同题目之间有明显的间隔，要能够容易地区分不同题目对应的实验内容
- 实验报告不做任何其他版式上的要求

本次实验不要求提交代码，只需提交实验报告即可，但实验报告应说明对代码的修改（如果有），以及丰富的截图和配套的文字说明（不能只有截图而无文字说明，或只有文字而无截图）。

建议开始之前，先查看第 4 部分环境搭建的内容。

使用 LLM：

- 可以使用 LLM 辅助完成实验，但需在实验报告开篇明确说明使用了哪个 LLM，在报告正文相应的地方仔细描述如何让 LLM 完成什么任务、获得了什么结果、结果是否需要人工修正、或者多轮迭代由 LLM 修正。建议将对话截图放到实验报告中。LLM 输出太长时可以不用截完整，另外通过文字简要描述即可。
- 如果（多轮迭代）使用 LLM 仍然没有获得一定程度上可用的结果（如返回结果中实际可用的部分不超过 3 成），则没有必要提及 LLM。

1. 实验目的

希望学生能对 Xinu 开发和通过 QEMU 运行调试操作系统内核有一个基本的掌握，为后续其他实验的开展打下基础。

2. 实验要求

2.1 修改 Xinu 代码，添加课件上并发执行的示例（“第一章 引论”第 22 页），观察在以下各种条件下的输出，并解释该输出的成因（**在不同环境下结果可能有所不同，根据自己观察的结果进行解释即可**）。注意：特别关注最开始的一段时间内的输出（如果输出太多、刷新太快，可设置循环次数）。

a main 函数进程与 sndA、sndB 进程的优先级一致；

b main 函数进程的优先级较低，而 sndA 和 sndB 的优先级较高且相等（调用 create 函数时的参数“20”即为优先级，数字越大，优先级越高）；

c **注意——额外要求（需要完成）**：从 Xinu 中寻找合适的函数，在 sndA 和 sndB 中进入 while 之前输出一串信息，包含学号、姓名和当前函数名（中文姓名如果显示为乱码的话，可以使用拼音）。

2.2 通过 QEMU 调试，在合适的地方设置断点，查看相关数据，然后据此画出 2.1.a 中自从进入 main 函数进程之后，上下文切换的流程，记录从哪个进程切换到哪个进程（从 gdb 中可以输出进程的名称，参看 struct procent 中的 prname 字段）、栈指针的变化（**注意**：与 3.7 c 中的问题一并回答）。

a 可以在 Word 中画，也可以在其他工具中画然后截图，也可以手画然后拍照。对于后两种形式，在原画面（其他工具或纸张上），写上“学号姓名”，不能是截图或拍照之后在图片上添加←**注意**

b **画图任务不能由 LLM 完成。**

2.3 任意选定一个进程的创建，通过调试的方式查看该进程的初始栈，观察 INITRET 和 funcaddr 在栈中的取值，并与 userret 和进程入口函数的实际值做对比。

a 实际值可通过在父进程中打印的形式输出，也可以在调试环境中获取（如在 gdb 中查看特定内存地址的数据）。

2.4 观察确认系统中最后一个普通进程自杀或正常结束之后（需禁止掉 Xinu 原有的 shell——如：将 main 函数中的代码都注释掉），系统的实际运行状态，描述此时进程表状态（进程表中有哪些进程，各自状态是什么，等等）、所执

行指令、当前进程的信息，提供截图+文字描述。

a 只需考虑普通进程优先级大于空进程即可，鼓励探索普通进程与空进程优先级相同的情况。

2.5 独立完成（可使用 LLM 辅助），可讨论，不能抄袭

3. 实验步骤

3.1 下载、编译 Xinu 源代码：准备好 Linux 编译环境（本机 Linux、虚拟机安装的 Linux 或 WSL，使用 QEMU 图形界面请参考 4.2 ；不使用 QEMU 图形界面请参考 4.3 ）

a. 对于 ARM 版的 Linux，可以参考 OBE“课件”中的“在 ARM 版上编译 Xinu”；

b. 对于 MacOS（Intel 或 Mx 芯片），也可以参考上述文档，或自行探索。

3.2 进入源代码 compile 目录，运行 `make`，如果成功生成 xinu.elf，表示编译成功；如果不确定，可以先执行 `make clean`，确保 xinu.elf 已被删除，再重新执行 `make`

3.3 通过 QEMU 运行 Xinu：

a 执行命令：`qemu-system-i386 -kernel xinu.elf`

b 在弹出的 QEMU 界面上按下 Ctrl+Alt+3，出现如下的界面（图 1），表明 Xinu 运行成功

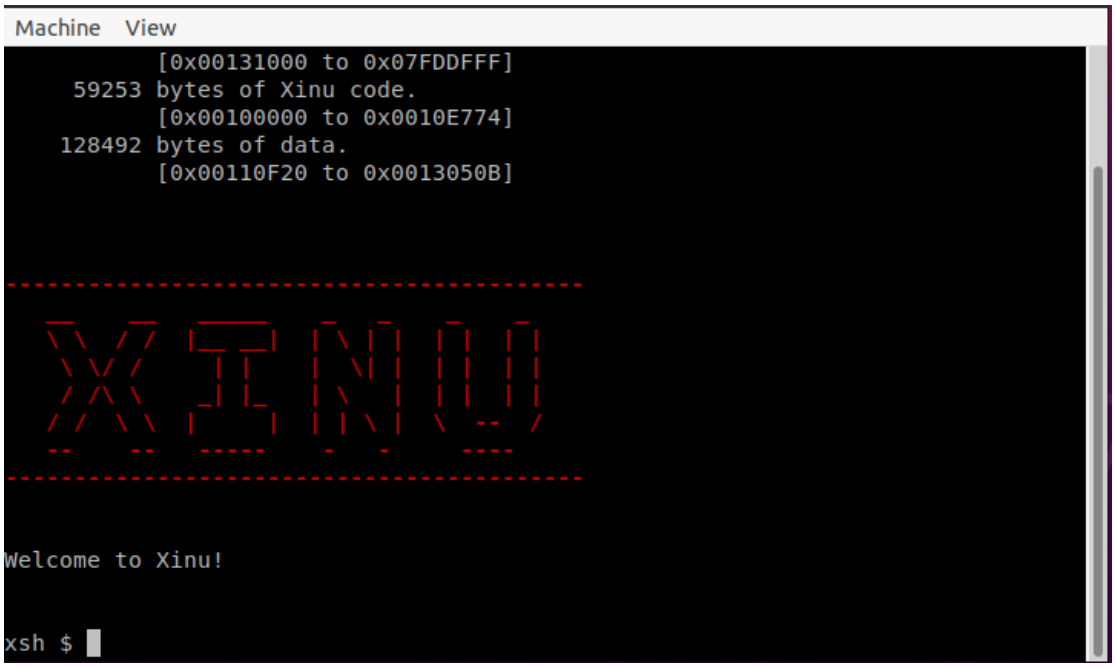

图 1

3.4 实验所要求的代码修改，在 system/main.c 中进行，将原有的 main 函数体用实验要求的 main 函数实现替换，并在该文件中添加 sndA 和 sndB 的实现

a main 进程的优先级在创建该进程时设置，请自行搜索代码位置

3.5 修改代码后，确保重新执行 make 命令；如果修改了.h 头文件，确保先执行 make clean 在执行 make

3.6 根据实验要求 2.1 修改代码，运行 Xinu 并观察输出结果

3.7 对于实验要求 2.2，打开两个终端窗口，其中一个以调试方式启动 QEMU 执行 Xinu，另一个执行 gdb，设置断点及单步调试

a 由于要求记录从进入 main 函数进程开始的上下文切换，所以在 gdb 中设置断点为 main 函数；又因要求记录进程的变化和栈顶指针的变化，所以在 ctxsw 函数中修改 ESP 寄存器的指令处设置断点（如果需要通过 currpid 来获知切换前后的进程 ID，则应在修改其值之前设置断点）

b 运行到 main 函数，暂停于断点处，在 gdb 中输入命令：`bt`，查看运行栈，此时应看到类似于如图 2 的信息：

图 2

c 在 gdb 中输入命令：`c`，以继续执行到下一个断点，输入命令：`bt`，查看函数调用栈，推断此时所在进程，并检查此时 ESP 寄存器的值；若此时暂停在 ctxsw 中的断点处，查看 ESP 的值，然后单步执行到 ctxsw 的 ret 指令处，再执行命令：`bt`，查看函数调用栈。注意观察刚改变 ESP 寄存器后的调用栈与执行到 ret 指令处的调用栈是否有区别，如果有，思考其原因。

d 重复 3.7 c 的操作，直到进程变化保持稳定（如永远只有进程 A 在输出，或进程 A 和进程 B 交替输出）。

4. 实验说明

主要内容（建议先仔细查看本节内容，然后再开始动手实验）：

- ✓ 构建编译环境、编译 Xinu、部分编译时可能会出现的问题；
- ✓ QEMU+GDB 调试 Xinu；
- ✓ 不使用图形化界面显示 Xinu（对于“键盘+显示器驱动”之外的实验均适用）；
- ✓ 使用 VNC 模式启动 QEMU，并通过 VNC 远程连接显示 Xinu（适用于不能在本地使用图形化 QEMU 的同学）。

4.1 Xinu 编译相关说明

a 确保已安装 gcc、make、gawk、flex、bison、libfl-dev，或根据编译代码的出错信息安装缺失的依赖

b 首次编译代码，若出现错误类似于：“make: *** No rule to make target 'binaries/start.o', needed by '../compile/xinu.elf'. Stop.”，重新执行一次 make 即可（不要 make clean）

c 本实验中，为方便观察，请确保 include/kernel.h 中 QUANTUM 为一个较小的数（如 2，表示时间片为 2 毫秒）

d 为便于调试，在 compile/Makefile 中为 CFLAGS 赋值处，加上 `-ggdb` 选线，以便在目标文件中保留调试信息——如果编译之后有修改，然后才想起添加调试选项，则先 make clean 清除原来的编译结果，甚至可以删除 compile 目录下的两个隐藏文件`.defs` 和`.deps`

4.2 QEMU 调试，以 Ubuntu 系统为例。

a 安装 qemu：`sudo apt install qemu-system-x86`

b 开启两个终端窗口，在一个窗口内执行命令：`qemu-system-i386 -S -s -kernel xinu.elf`

- ➢ -S：在启动时就挂起 CPU，等待 gdb 连接
- ➢ -s：等价于-gdb tcp::1234，表示打开默认的 1234 这个 gdb 调试端口，也可以通过显式使用-gdb tcp::X 的方式指定调试端口

c 在 QEMU 窗口按下 Ctrl+Alt+3 之后，如图 3 所示，表明 Xinu 并未完成启动，在等待调试器的命令：

图 3

d　在另一个终端窗口内执行命令：gdb，应看到类似于如图 4 的信息：

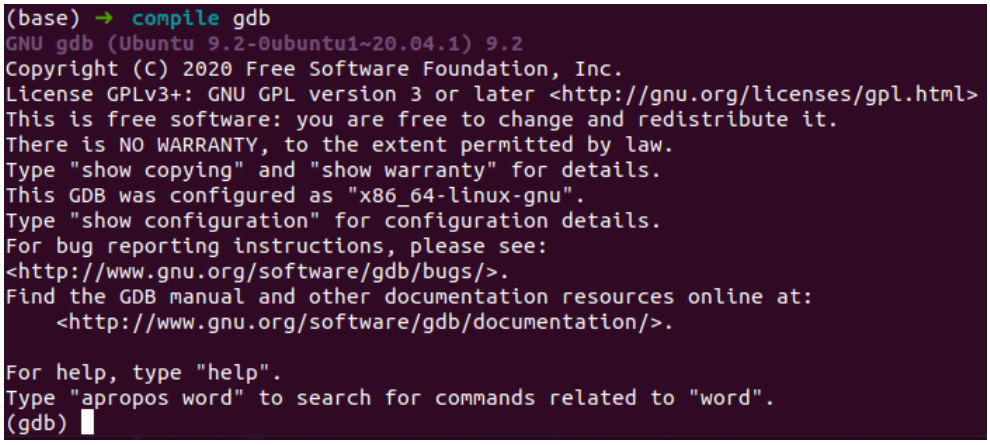

图 4

e　输入：target remote :1234，可观察到如图 5 所示的内容

- 如果手动指定了调试端口 X，则输入命令：target remote :X
- :1234 等价于 localhost:1234

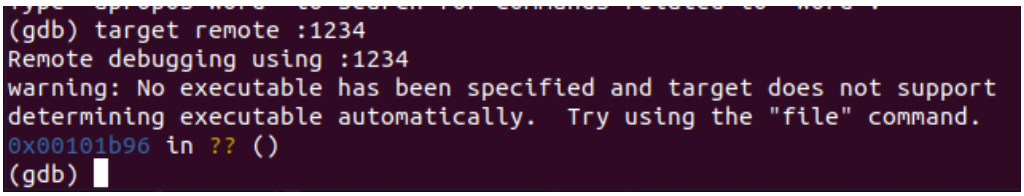

图 5

f　在 gdb 中输入：`c`，应观察到 QEMU 能够正常显示输出，表明 Xinu 已完成启动，正常运行

- c 为 continue 的缩写

g　在 4.2 f 输入“c”之前，可先加载符号文件，即 xinu.elf，便于以源码位置设置断点，命令为：`file xinu.elf`（见图 6）

```
(gdb) file xinu.elf
A program is being debugged already.
Are you sure you want to change the file? (y or n) y
Reading symbols from xinu.elf...
```

图 6

h　加载符号文件之后，可设置断点，命令：`b main`，在 main 函数处设置断点（图 7），然后输入命令：`c`，可观察到 QEMU 中运行 Xinu（以原始版本为例，非实验要求修改代码后的版本）有一定输出，但并未运行倒 shell（图 8）：

- 命令 bt 为 backtrace 的缩写

```
(gdb) b main
Breakpoint 1 at 0x102423: file ../system/main.c, line 6.
(gdb) c
Continuing.

Breakpoint 1, main () at ../system/main.c:6
6       {
(gdb)
```

图 7

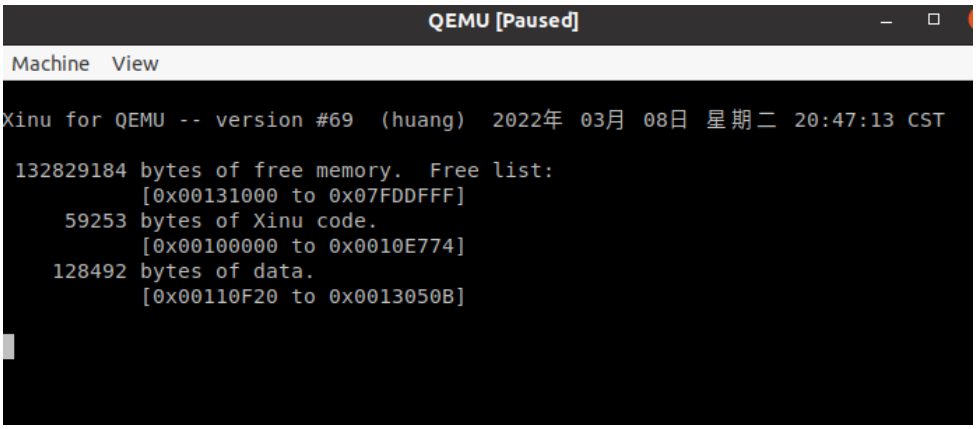

图 8

4.3 QEMU 无图形化模式，以 WSL 中无图形化模式为例（较新的 WSL 也能支持图形化模式的 QEMU）。

a　在 WSL 中安装 qemu-system-x86

b　开启两个 WSL 终端，在其中一个输入命令：`qemu-system-i386 -`

`nographic` `-serial mon:stdio` `-S -s -kernel xinu.elf`，意为使用非图形化模式，标准 IO 定向为串口 IO，其余的选项与 Ubuntu 下一致

c　在另一个 WSL 终端中执行 gdb，具体操作参考 4.2

d　第一个终端窗口中 Xinu 运行起来的效果如图 9 所示

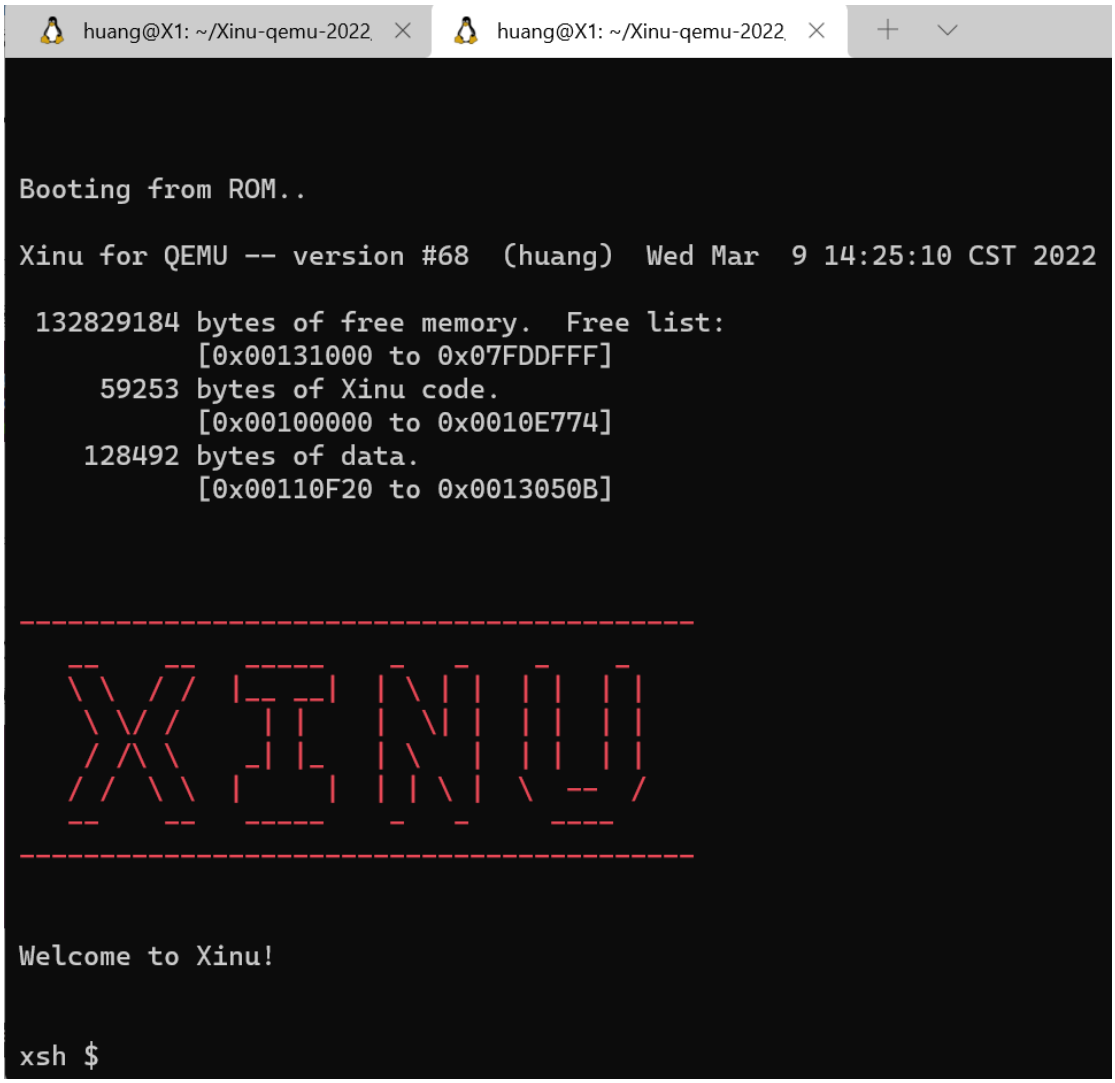

图 9

e　在图 9 所示界面，可通过 Ctrl+a h（即同时按下 Ctrl 键和 A 键，然后释放并迅速按下 h），查看辅助命令，如图 10 所示：

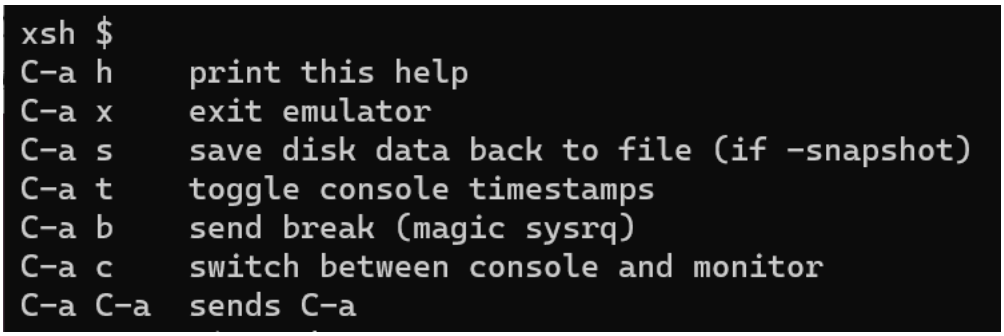

图 10

f　按下 Ctrl+a x，即可在非图形化模式下退出 QEMU

g　注意：在 Windows 11+WSL 2 中，执行 `qemu-system-i386 -kernel xinu.elf`，也会弹出一个 QEMU 窗口，如图 11 所示

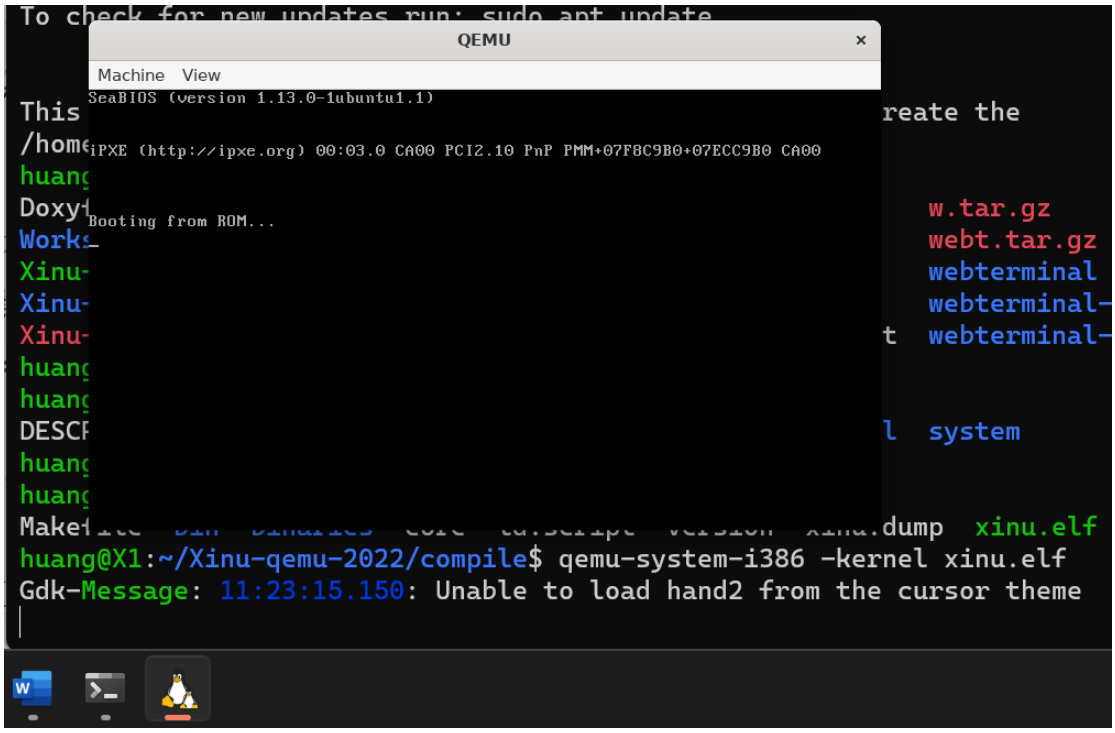

图 11

4.4 使用 VNC

如果 WSL 或其他 Linux 环境（如远程服务器）不能提供图形界面的 QEMU，可以尝试通过 VNC 的方式连接 QEMU。

a　以 VNC 启动 QEMU：`qemu-system-i386 -kernel xinu.elf -vnc :2`，添加-vnc 选项，后边的“:2”表示 2 号显示器，也可以使用尚未被占用的其他编号（通常是一个较小的数字）

b　通过 VNC 连接，可以使用 VNC Viewer[1]或其他支持 VNC 的工具，然后通过 ip:2 的方式连接。其中，ip 是 localhost（如使用本机 WSL）或远程服务器 IP，2 为上一步的显示器编号。效果如图 12 所示。

c　通过 VNC 连接时，同样可以使用调试模式，如图 13 所示。

[1] https://www.realvnc.com/en/connect/download/viewer/

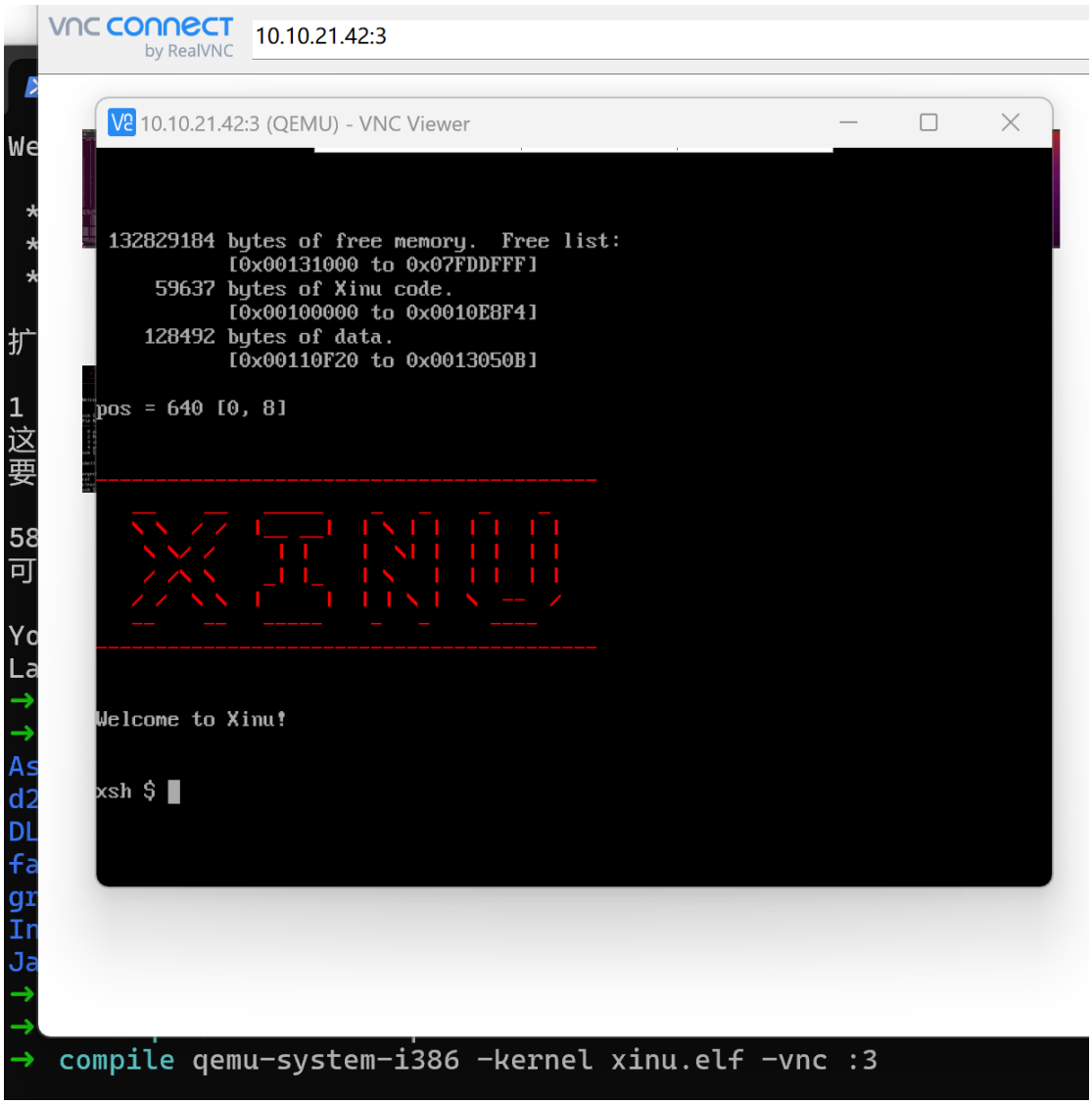
VNC connect by RealVNC
10.10.21.42:3
10.10.21.42:3 (QEMU) - VNC Viewer
132829184 bytes of free memory. Free list:
[0x00131000 to 0x07FDDFFF]
59637 bytes of Xinu code.
[0x00100000 to 0x0010E8F4]
128492 bytes of data.
[0x00110F20 to 0x0013050B]
pos = 640 [0, 8]
Welcome to Xinu!
xsh $
compile qemu-system-i386 -kernel xinu.elf -vnc :3

图 12

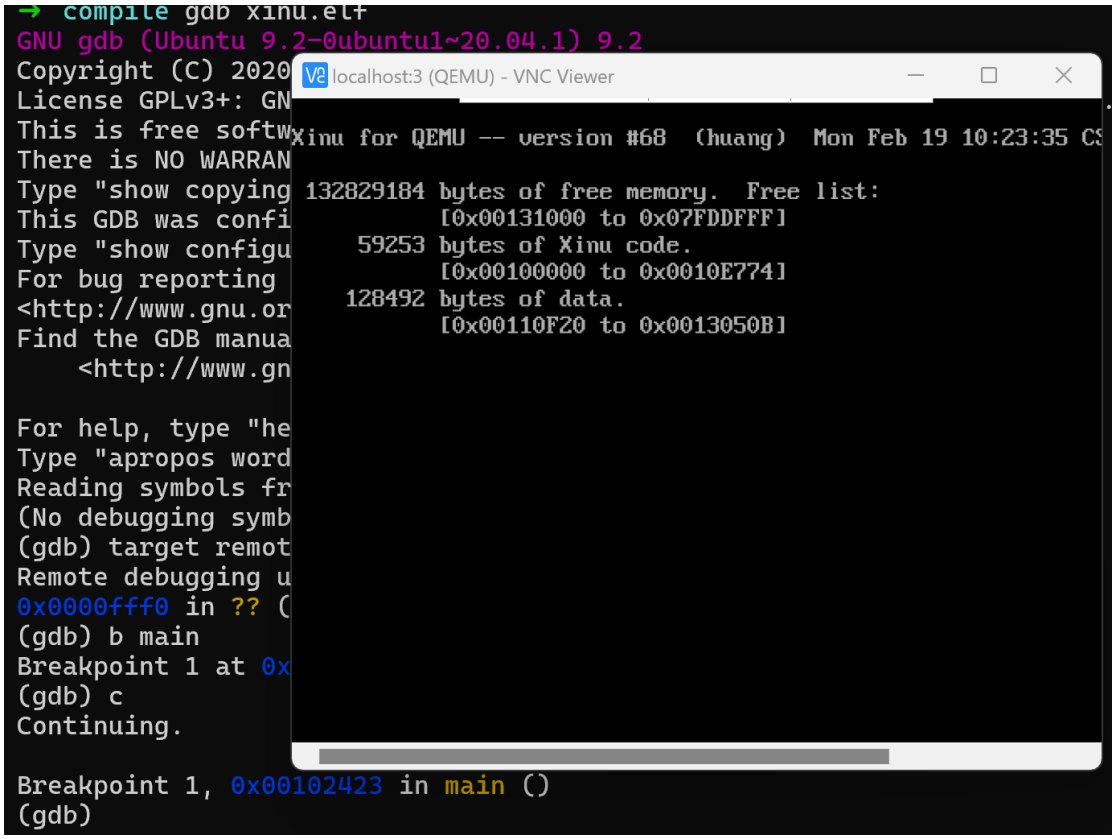

图 13

5. 思维扩展

（不要求在实验报告中作答）

回顾调度与上下文切换的基本原则和流程，以及进程创建的操作和进程栈的初始化，并将其与上下文切换相联系。

仔细察看 OpenHarmony 中 LiteOS-M 和 LiteOS-A 的任务调度/进程调度的代码实现，描述其具体流程（也可以执行下载编译、模拟执行、调试查看，以进一步深入理解其过程）。

https://gitcode.com/openharmony/kernel_liteos_m/blob/master/kernel/src/los_sched.c

https://gitcode.com/openharmony/kernel_liteos_a/blob/master/kernel/base/sched/los_sched.c